

CO2-AT2-SCHEMA ANALYSIS

Course Code: DSA0504

**Course Title: Query Processing for
Data Science**

CO Assessed: CO2

**Assessment: Tool 2 –Schema
Analysis**

Weightage: 40

Total Marks: 25 Marks

CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)

Student Name: Jeya Harshini R

Reg. No.:192424051

Section / Batch:2024

Date of Submission:3/8/26

Faculty In-charge: Dr. R.Mahesh Muthulakshmi

Project Mode: Individual Submission

QUESTION 1 – STUDENT DATABASE DESIGN AND CONNECTIVITY

Question

A college wants to automate the management of student records. Design a PostgreSQL database containing a **Student** table with appropriate fields and a primary key. Write a Python program to connect to the PostgreSQL database, create the table if it does not exist, insert at least five student records, and display all the records stored in the table.

1. Aim

To establish connectivity between Python and PostgreSQL using the **psycopg2** library, create a Student table with appropriate attributes and a primary key, insert sample student records into the database, and retrieve all the stored records successfully.

2. Objective

- To establish connectivity between Python and PostgreSQL.
 - To create a Student table with a primary key.
 - To insert multiple student records into the PostgreSQL database.
 - To retrieve and display all student records.
 - To understand database connectivity using the psycopg2 library.
-

3. Software Requirements

Software	Version
Operating System	Windows 10/11
Python	Python 3.x
PostgreSQL	PostgreSQL 18 (or installed version)
pgAdmin	pgAdmin 4
Python Library	psycopg2

4. Database Connectivity

Database connectivity is the process of establishing communication between an application and a database management system. It enables applications to create, retrieve, update, and delete data efficiently.

In this experiment, Python is connected to the PostgreSQL database using the **psycopg2** library. The `connect()` method establishes a connection with the PostgreSQL server using the host name, database name, username, password, and port number. After establishing the connection, a cursor object is created to execute SQL commands. The `commit()` method permanently saves all changes made to the database, while the `close()` method safely terminates the database connection after all operations are completed.

5. Database Design

Database Name

college

Table Name : Student

Field Name	Data Type	Constraint	Description
Student_ID	INTEGER	PRIMARY KEY	Unique Student ID
Student_Name	VARCHAR(50)	NOT NULL	Name of the Student
Department	VARCHAR(50)	NOT NULL	Department Name
Age	INTEGER	NOT NULL	Student Age
Marks	DECIMAL(5,2)	-	Marks Obtained

6. Schema Diagram

SCHEMA DIAGRAM

Database Name : college

STUDENT		
Student_ID	: INTEGER	PRIMARY KEY
Student_Name	: VARCHAR(50)	NOT NULL
Department	: VARCHAR(50)	NOT NULL
Age	: INTEGER	NOT NULL
Marks	: DECIMAL(5,2)	—

- **Primary Key** : Student_ID uniquely identifies each student.
- **Foreign Key** : None
- **Relationship** : Single Table – No relationship exists.

Single Table Database – No Foreign Key

Relationship

- Single Table Database
- No Foreign Key Relationship

7. Algorithm

Step 1: Import the psycopg2 library.

Step 2: Establish a connection with the PostgreSQL database.

Step 3: Create a cursor object.

Step 4: Create the Student table if it does not already exist.

Step 5: Insert five student records into the table.

Step 6: Commit the transaction.

Step 7: Retrieve all records using the SELECT statement.

Step 8: Display the records.

Step 9: Close the cursor and database connection.

9. Program Explanation

The Python program begins by importing the **psycopg2** library, which enables communication between Python and PostgreSQL. A connection is established using the database credentials such as host name, database name, username, password, and port number.

After successful connection, a cursor object is created to execute SQL commands. The Student table is created with Student_ID as the primary key. Five sample student records are inserted using the INSERT statement. The commit() function permanently stores the inserted records in the database.

Finally, the SELECT statement retrieves all the student records, which are displayed on the console. The cursor and database connection are then closed to release system resources.

10. Code

```
import psycopg2

# Connect to PostgreSQL
conn = psycopg2.connect(
    host="localhost",
    database="college",
    user="postgres",
    password="jeyah812",
    port="5432"
)

print("Connected Successfully!")

cur = conn.cursor()

# Delete old table if it exists
cur.execute("DROP TABLE IF EXISTS Student")

# Create new Student table
cur.execute("""
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Student_Name VARCHAR(50) NOT NULL,
    Department VARCHAR(50) NOT NULL,
    Age INT NOT NULL,
    Marks DECIMAL(5,2)
)

```

```

""")

# Remove existing records (optional, to avoid duplicate primary keys)
cur.execute("DELETE FROM Student")

# Insert student records
students = [
    (101, "Alice", "AI", 20, 91.50),
    (102, "Bob", "CSE", 21, 85.00),
    (103, "Charlie", "ECE", 19, 88.50),
    (104, "David", "IT", 20, 79.00),
    (105, "Eva", "AI", 21, 95.20)
]

cur.executemany(
    "INSERT INTO Student (Student_ID, Student_Name, Department, Age, Marks) VALUES (%s, %s, %s, %s, %s)",
    students
)

conn.commit()

print("\nFive student records inserted successfully.\n")

# Display all records
cur.execute("SELECT * FROM Student")

rows = cur.fetchall()

print("Student Records")
print("-" * 65)
print(f"{'ID':<8}{'Name':<15}{'Department':<15}{'Age':<8}{'Marks'}")
print("-" * 65)

for row in rows:
    print(f"{'row[0]':<8}{'row[1]':<15}{'row[2]':<15}{'row[3]':<8}{'row[4]}")

# Close connection
cur.close()
conn.close()

print("\nDatabase connection closed.")

```

12. Output Screenshot

```
===== RESTART:
Connected Successfully!

Five student records inserted successfully.

Student Records
-----
ID      Name      Department  Age  Marks
-----
101     Alice      AI          20   91.50
102     Bob        CSE         21   85.00
103     Charlie    ECE         19   88.50
104     David      IT          20   79.00
105     Eva        AI          21   95.20

Database connection closed.
```

13. Database Analysis

Primary Key

- Student_ID

Foreign Key

- None

Constraints Used

- PRIMARY KEY
- NOT NULL

Operations Performed

- CREATE TABLE
- INSERT
- SELECT

Connectivity Analysis

The database connection was established successfully using the psycopg2 library. SQL commands were executed through the cursor object. The commit() function ensured permanent storage of records, while the SELECT query successfully retrieved all stored data from the Student table.

14. Advantages

- Provides secure storage of student information.
- Eliminates manual record maintenance.
- Enables quick retrieval of student details.
- Reduces data redundancy.
- Supports efficient database management.

15. Applications

- College Student Management System
- University Information System
- School Administration
- Academic Record Management
- Educational ERP Systems

17. Conclusion

The Student Database Management System was successfully implemented using Python and PostgreSQL. The Student table was created with appropriate attributes and constraints. Five student records were inserted successfully into the PostgreSQL database and retrieved using the SELECT statement. The experiment demonstrated successful database connectivity and execution of SQL operations through Python.

QUESTION 2 – LIBRARY MANAGEMENT DATABASE

Question

A library plans to maintain information about its books using a PostgreSQL database. Design a **Book** table containing suitable attributes such as **Book ID, Title, Author, Publisher, and Price**. Write a Python program to establish a connection with the PostgreSQL database, create the table, insert sample records, update the price of a selected book, and retrieve the updated information.

1. Aim

To establish connectivity between Python and PostgreSQL using the **psycopg2** library, create a **Book** table with appropriate attributes and a primary key, insert sample book records, update the price of a selected book, and retrieve the updated information successfully.

2. Objective

- To establish connectivity between Python and PostgreSQL.
 - To create a Book table with appropriate fields.
 - To insert sample book records into the database.
 - To update the price of a selected book.
 - To retrieve and display the updated records.
 - To understand CRUD operations using Python and PostgreSQL.
-

3. Software Requirements

Software	Version
Operating System	Windows 10/11
Python	Python 3.x
PostgreSQL	PostgreSQL 18

Software	Version
pgAdmin	pgAdmin 4
Python Library	psycopg2

4. Database Connectivity

Database connectivity enables communication between Python applications and PostgreSQL databases. In this experiment, the **psycopg2** library is used to establish the connection. The connect() method connects Python to PostgreSQL using the database credentials. A cursor object executes SQL commands such as CREATE, INSERT, UPDATE, and SELECT. The commit() method permanently saves all modifications made to the database, and the connection is closed using the close() method after successful execution.

5. Database Design

Database Name

college

Table Name : Book

Field Name	Data Type	Constraint	Description
Book_ID	INTEGER	PRIMARY KEY	Unique Book ID
Title	VARCHAR(100)	NOT NULL	Title of the Book
Author	VARCHAR(50)	NOT NULL	Author Name
Publisher	VARCHAR(50)	NOT NULL	Publisher Name
Price	DECIMAL(8,2)	NOT NULL	Book Price

6. Schema Diagram

SCHEMA DIAGRAM

Database Name : college

BOOK			
Field Name	Data Type	Constraint	Description
Book_ID	INTEGER	PRIMARY KEY	Unique Book ID
Title	VARCHAR(100)	NOT NULL	Title of the Book
Author	VARCHAR(50)	NOT NULL	Author Name
Publisher	VARCHAR(50)	NOT NULL	Publisher Name
Price	DECIMAL(8,2)	NOT NULL	Book Price

BOOK	
<u>PK</u>	Book_ID (INTEGER)
	Title (VARCHAR(100))
	Author (VARCHAR(50))
	Publisher (VARCHAR(50))
	Price (DECIMAL(8,2))

- ♦ **Primary Key** : Book_ID uniquely identifies each book.
- ♦ **Foreign Key** : None
- ♦ **Relationship** : Single Table – No relationship exists.

Single Table Database – No Foreign Key

Relationship

- Single Table Database
- No Foreign Key Relationship

7. Algorithm

Step 1: Import the psycopg2 library.

Step 2: Establish a connection with the PostgreSQL database.

Step 3: Create the Book table if it does not already exist.

Step 4: Insert sample book records.

Step 5: Update the price of a selected book.

Step 6: Retrieve the updated records.

Step 7: Display the records.

Step 8: Close the cursor and database connection.

8. Python Program

```
import psycopg2

# Connect to PostgreSQL
conn = psycopg2.connect(
    host="localhost",
    database="college",
    user="postgres",
    password="jeyah812",
    port="5432"
)

print("Connected Successfully!")

cur = conn.cursor()

# Delete old table if it exists
cur.execute("DROP TABLE IF EXISTS Book")

# Create Book table
cur.execute("""
CREATE TABLE Book(
    Book_ID INT PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    Author VARCHAR(50) NOT NULL,
    Publisher VARCHAR(50) NOT NULL,
    Price DECIMAL(8,2) NOT NULL
)
""")

# Insert sample book records
books = [
    (101, "Python Programming", "Reema", "McGraw Hill", 600.00),
    (102, "Database Systems", "Navathe", "Pearson", 780.00),
    (103, "Java Fundamentals", "Herbert", "Oracle Press", 550.00),
    (104, "AI Basics", "Stuart", "Wiley", 720.00),
    (105, "Data Science", "Han", "Springer", 900.00)
]

cur.executemany(
    "INSERT INTO Book VALUES (%s,%s,%s,%s,%s)",
    books
)
```

```

conn.commit()

print("\nBook records inserted successfully.\n")

# Update the price of Book_ID = 101
cur.execute("""
UPDATE Book
SET Price = 650.00
WHERE Book_ID = 101
""")

conn.commit()

print("Book price updated successfully.\n")

# Display updated records
cur.execute("SELECT * FROM Book")

rows = cur.fetchall()

print("Updated Book Records")
print("-" * 80)
print(f'{ID':<8}{Title':<25}{Author':<15}{Publisher':<18}{Price}')
print("-" * 80)

for row in rows:
    print(f'{row[0]:<8}{row[1]:<25}{row[2]:<15}{row[3]:<18}{row[4]}')

cur.close()
conn.close()

print("\nDatabase connection closed.")

```

9. Program Explanation

The program begins by importing the **psycopg2** library to establish communication with PostgreSQL. After successfully connecting to the database, a cursor object is created to execute SQL commands. A Book table is created with Book_ID as the primary key. Sample book records are inserted into the table. The price of a selected book is updated using the SQL UPDATE statement. Finally, the updated records are retrieved using the SELECT statement and displayed on the console before closing the database connection.

12. Output Screenshot

```
===== RESTART: C:\Users\haisi
Connected Successfully!

Book records inserted successfully.

Book price updated successfully.

Updated Book Records
-----
ID      Title                Author    Publisher    Price
-----
102     Database Systems       Navathe   Pearson      780.00
103     Java Fundamentals      Herbert   Oracle Press  550.00
104     AI Basics              Stuart    Wiley         720.00
105     Data Science           Han       Springer     900.00
101     Python Programming     Reema     McGraw Hill   650.00
Database connection closed.
```

13. Database Analysis

Primary Key

- Book_ID

Foreign Key

- None

Constraints Used

- PRIMARY KEY
- NOT NULL

Operations Performed

- CREATE TABLE
- INSERT
- UPDATE
- SELECT

Connectivity Analysis

The PostgreSQL database was successfully connected using the psycopg2 library. The Book table was created, sample records were inserted, and the UPDATE statement modified

the price of the selected book. The updated information was retrieved using the SELECT statement, demonstrating successful execution of CRUD operations.

14. Advantages

- Maintains book records efficiently.
 - Simplifies updating book information.
 - Enables quick retrieval of book details.
 - Reduces manual record maintenance.
 - Provides secure storage of library data.
-

15. Applications

- Library Management System
 - School Libraries
 - College Libraries
 - Digital Library Systems
 - Book Inventory Management
-

17. Conclusion

The Library Management Database was successfully implemented using Python and PostgreSQL. The Book table was created with appropriate fields and constraints. Sample book records were inserted, the price of a selected book was updated successfully, and the updated records were displayed. The experiment demonstrates successful Python–PostgreSQL database connectivity and CRUD operations.

QUESTION 3 – HOSPITAL PATIENT DATABASE

Question

A hospital requires a PostgreSQL database to manage patient information. Design a **Patient** table containing suitable fields and constraints. Write a Python program to connect to the PostgreSQL database, create the table, insert patient records, update the contact number of a specified patient, delete a discharged patient's record, and display the remaining records.

1. Aim

To establish connectivity between Python and PostgreSQL using the **psycopg2** library, create a **Patient** table with appropriate attributes and constraints, insert patient records, update patient contact information, delete a discharged patient's record, and display the remaining records.

2. Objective

- To establish connectivity between Python and PostgreSQL.
 - To create a Patient table with appropriate fields and constraints.
 - To insert patient records into the database.
 - To update the contact number of a patient.
 - To delete a discharged patient's record.
 - To retrieve and display the remaining patient records.
-

3. Software Requirements

Software	Version
Operating System	Windows 10/11
Python	Python 3.x
PostgreSQL	PostgreSQL 18

Software	Version
pgAdmin	pgAdmin 4
Python Library	psycopg2

4. Database Connectivity

Database connectivity allows Python applications to communicate with PostgreSQL databases for storing, retrieving, updating, and deleting information. In this experiment, the **psycopg2** library is used to establish the database connection. The connect() method creates the connection, the cursor object executes SQL commands, commit() saves all changes permanently, and close() safely terminates the database connection after execution.

5. Database Design

Database Name

college

Table Name : Patient

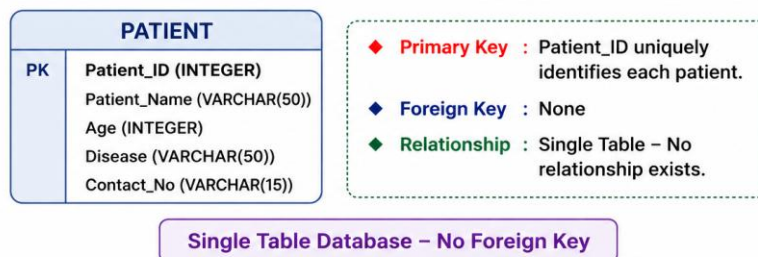
Field Name	Data Type	Constraint	Description
Patient_ID	INTEGER	PRIMARY KEY	Unique Patient ID
Patient_Name	VARCHAR(50)	NOT NULL	Name of the Patient
Age	INTEGER	NOT NULL	Patient Age
Disease	VARCHAR(50)	NOT NULL	Disease Name
Contact_No	VARCHAR(15)	NOT NULL	Contact Number

6. Schema Diagram

SCHEMA DIAGRAM

Database Name : college

PATIENT			
Field Name	Data Type	Constraint	Description
Patient_ID	INTEGER	PRIMARY KEY	Unique Patient ID
Patient_Name	VARCHAR(50)	NOT NULL	Name of the Patient
Age	INTEGER	NOT NULL	Age of the Patient
Disease	VARCHAR(50)	NOT NULL	Disease Name
Contact_No	VARCHAR(15)	NOT NULL	Contact Number



Relationship

- Single Table Database
- No Foreign Key Relationship

7. Algorithm

- Step 1:** Import the pycpg2 library.
- Step 2:** Establish a connection with the PostgreSQL database.
- Step 3:** Create the Patient table.
- Step 4:** Insert sample patient records.
- Step 5:** Update the contact number of a specified patient.
- Step 6:** Delete the discharged patient's record.
- Step 7:** Retrieve and display the remaining patient records.
- Step 8:** Close the cursor and database connection.

8. Python Program

```
import pycpg2
```

```

# Connect to PostgreSQL
conn = psycopg2.connect(
    host="localhost",
    database="college",
    user="postgres",
    password="your_password",
    port="5432"
)

print("Connected Successfully!")

cur = conn.cursor()

# Delete old table if it exists
cur.execute("DROP TABLE IF EXISTS Patient")

# Create Patient table
cur.execute("""
CREATE TABLE Patient(
    Patient_ID INT PRIMARY KEY,
    Patient_Name VARCHAR(50) NOT NULL,
    Age INT NOT NULL,
    Disease VARCHAR(50) NOT NULL,
    Contact_No VARCHAR(15) NOT NULL
)
""")

# Insert patient records
patients = [
    (101, "Ramesh", 45, "Fever", "9876543210"),
    (102, "Priya", 30, "Diabetes", "9876543211"),
    (103, "Arun", 50, "Asthma", "9876543212"),
    (104, "Divya", 28, "Migraine", "9876543213"),
    (105, "Karthik", 40, "Fracture", "9876543214")
]

cur.executemany(
    "INSERT INTO Patient VALUES (%s,%s,%s,%s,%s)",
    patients
)

conn.commit()

print("\nPatient records inserted successfully.\n")

# Update contact number
cur.execute("""
UPDATE Patient
SET Contact_No='99999999999'
WHERE Patient_ID=102
""")

conn.commit()

```

```

print("Patient contact number updated successfully.\n")

# Delete discharged patient
cur.execute("""
DELETE FROM Patient
WHERE Patient_ID=105
""")

conn.commit()

print("Discharged patient record deleted successfully.\n")

# Display remaining records
cur.execute("SELECT * FROM Patient")

rows = cur.fetchall()

print("Remaining Patient Records")
print("-"*80)
print(f"{'ID':<8}{'Name':<15}{'Age':<8}{'Disease':<15}{'Contact'}")
print("-"*80)

for row in rows:
    print(f"{'row[0]':<8}{'row[1]':<15}{'row[2]':<8}{'row[3]':<15}{'row[4]'}")

cur.close()
conn.close()

print("\nDatabase connection closed.")

```

9. Program Explanation

The program imports the **psycopg2** library and establishes a connection to the PostgreSQL database. A Patient table is created with **Patient_ID** as the primary key. Five patient records are inserted into the table. The contact number of a specified patient is updated using the UPDATE statement, and a discharged patient's record is removed using the DELETE statement. Finally, the remaining records are retrieved using the SELECT statement and displayed on the console before closing the database connection.

12. Output Screenshot

```
===== RESTART: C:\Us
Connected Successfully!

Patient records inserted successfully.

Patient contact number updated successfully.

Discharged patient record deleted successfully.

Remaining Patient Records
-----
ID      Name      Age      Disease      Contact
-----
101     Ramesh     45      Fever       9876543210
103     Arun       50      Asthma      9876543212
104     Divya      28      Migraine    9876543213
102     Priya      30      Diabetes    9999999999

Database connection closed.
|
```

13. Database Analysis

Primary Key

- Patient_ID

Foreign Key

- None

Constraints Used

- PRIMARY KEY
- NOT NULL

Operations Performed

- CREATE TABLE
- INSERT
- UPDATE
- DELETE
- SELECT

Connectivity Analysis

The PostgreSQL database was successfully connected using the psycopg2 library. SQL commands were executed using the cursor object. Patient records were inserted, updated, deleted, and retrieved successfully. The commit() function permanently saved all modifications made to the database.

14. Advantages

- Efficient storage of patient information.
 - Easy updating of patient records.
 - Supports deletion of discharged patient records.
 - Reduces manual paperwork.
 - Improves hospital data management.
-

15. Applications

- Hospital Management Systems
 - Clinic Management Software
 - Medical Record Systems
 - Healthcare Information Systems
 - Patient Registration Systems
-

17. Conclusion

The Hospital Patient Database was successfully implemented using Python and PostgreSQL. The Patient table was created with appropriate fields and constraints. Patient records were inserted, updated, and deleted successfully. The remaining records were retrieved and displayed, demonstrating successful Python–PostgreSQL database connectivity and CRUD operations.
